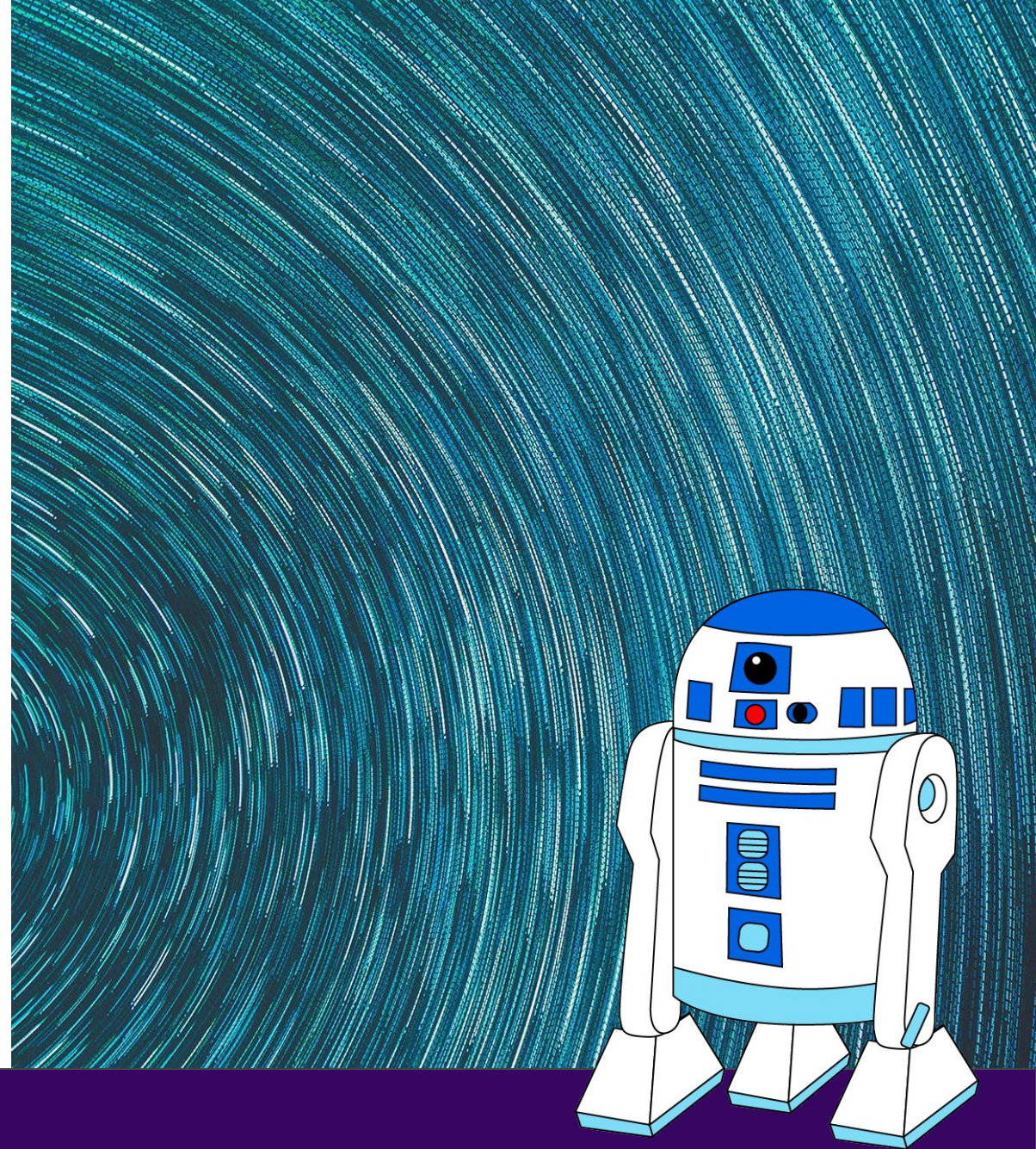


CIS 421/521:
ARTIFICIAL INTELLIGENCE

Neural Networks and Neural LMs

Jurafsky and Martin Chapters 7 and 9

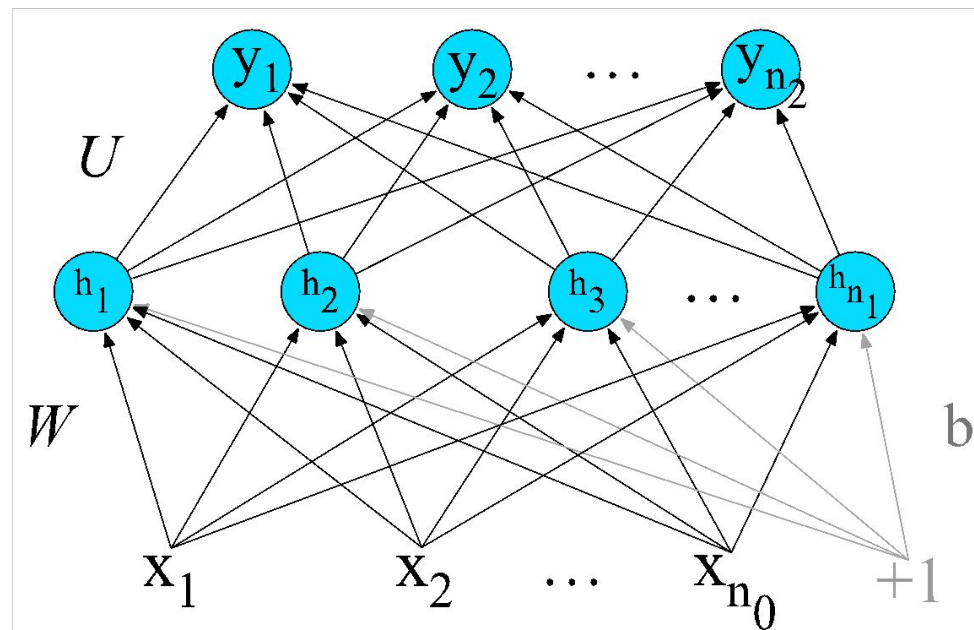


Review: Feed-Forward Neural Network

The simplest kind of is the **Feed-Forward Neural Network**

Multilayer network, all units are usually **fully-connected**, and **no cycles**.

The outputs from each layer are passed to units in the next higher layer, and no outputs are passed back to lower layers.

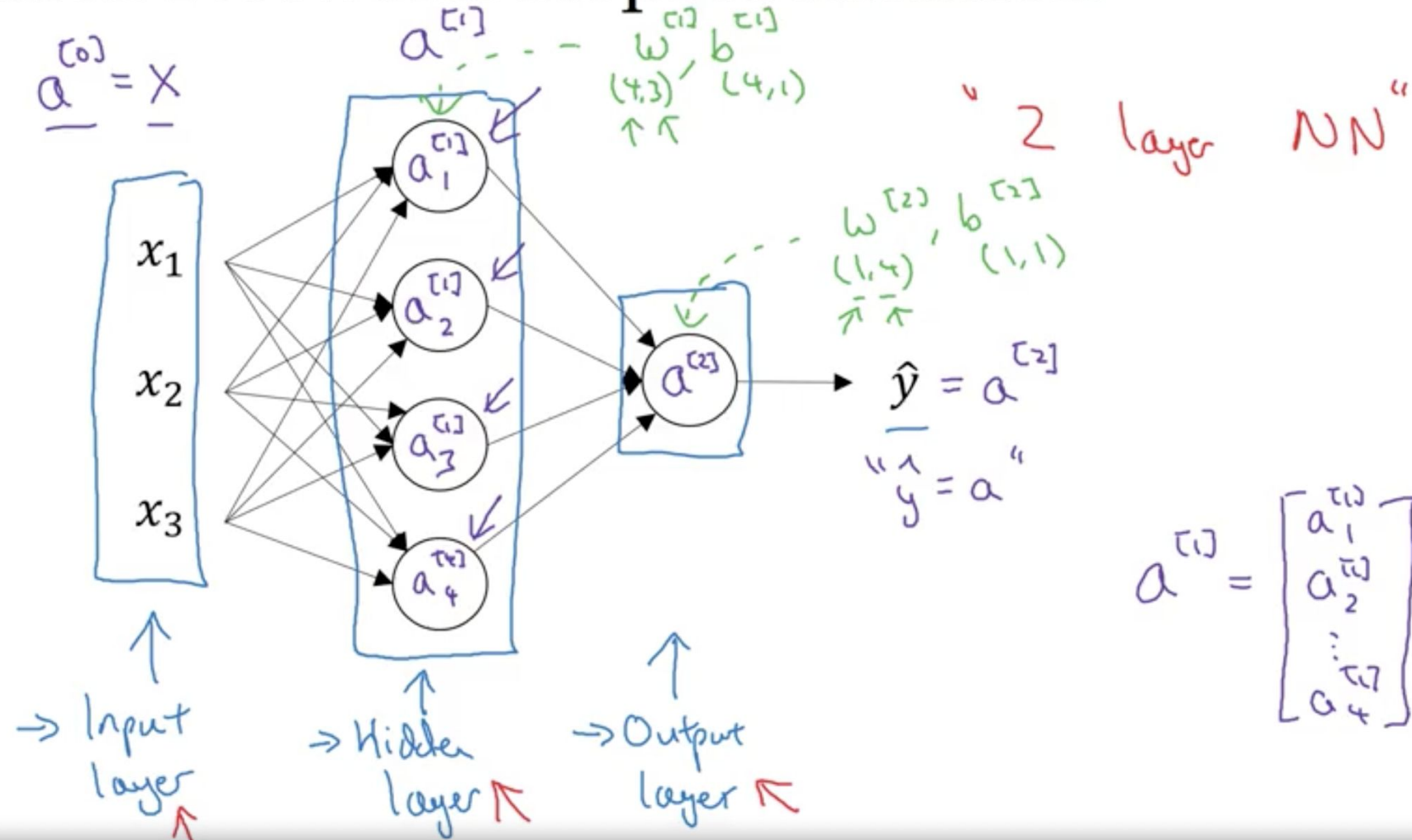


Layer 2 (output layer)

Layer 1 (hidden layer)

Layer 0 (input layer)

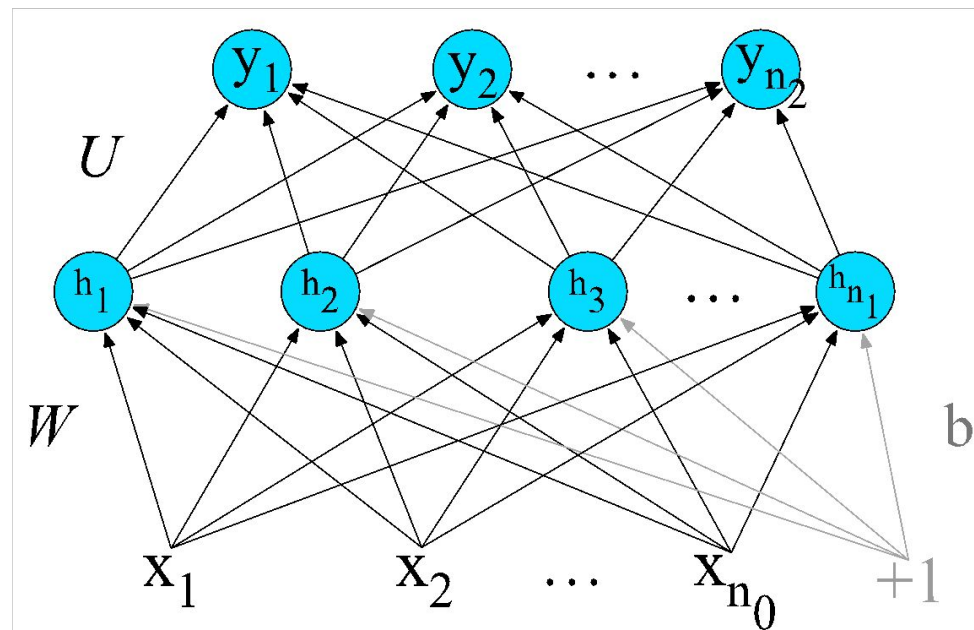
Neural Network Representation



Review: Feed-Forward Neural Network

A single hidden unit has parameters $\mathbf{w}$ (the weight vector) and $\mathbf{b}$ (the bias scalar).

We represent the parameters for the **entire hidden layer** by combining the weight vector $\mathbf{w}_i$ and bias $\mathbf{b}_i$ for each unit i into a single weight matrix $\mathbf{W}$ and a single bias vector $\mathbf{b}$ for the whole layer.



Layer 2 (output layer)

Layer 1 (hidden layer)

Layer 0 (input layer)

Review: Feed-Forward Neural Network

The advantage of using a single matrix $\mathbf{W}$ for the weights of the entire layer is the hidden layer computation can be done efficiently with simple matrix operations.

The computation has three steps:

1. multiplying the weight matrix by the input vector x ,
2. adding the bias vector b , and
3. applying the activation function g (such as Sigmoid)

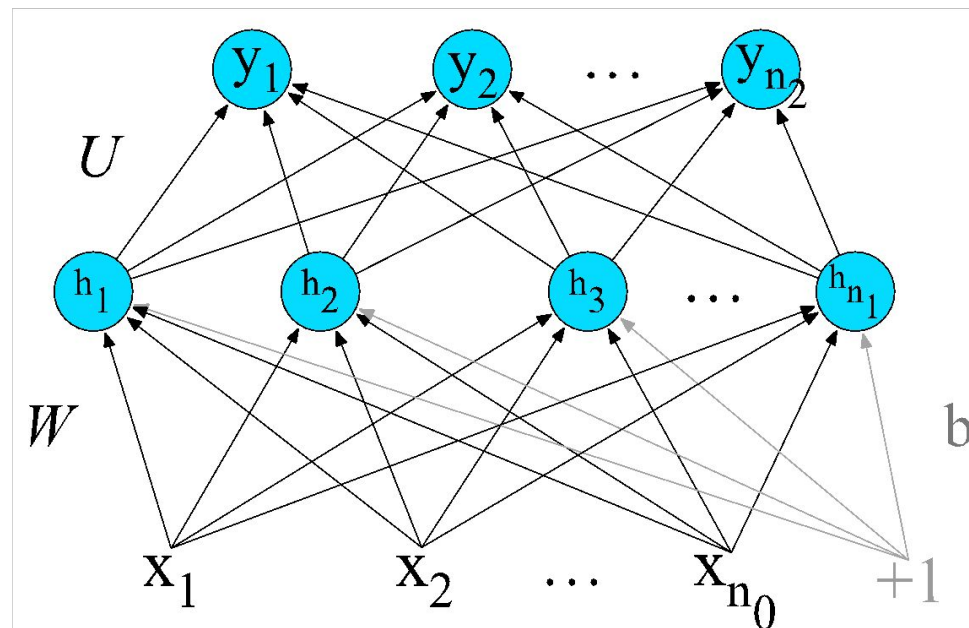
The output of the hidden layer, the vector h , is thus the following, using the sigmoid function σ :

$$h = \sigma(Wx+b)$$

Review: Feed-Forward Neural Network

Like the hidden layer, the output layer has a weight matrix $\mathbf{U}$.
Its weight matrix is multiplied by its input vector ($\mathbf{h}$) to produce the intermediate output $\mathbf{z}$.

$$\mathbf{z} = \mathbf{U}\mathbf{h}$$



Layer 2 (output layer)

Layer 1 (hidden layer)

Layer 0 (input layer)

Review: Feed-Forward Neural Network

Here are the final equations for a feedforward network with a single hidden layer, which takes an input vector x , outputs a probability distribution y , and is parameterized by weight matrices W and U and a bias vector b :

$$h = \sigma(Wx + b)$$

$$z = Uh$$

$$y = \text{softmax}(z)$$

Like with logistic regression, softmax normalizes the output and turns it into a probability distribution.

Training Neural Nets



Cross-Entropy Loss

If the neural network is a binary classifier with a sigmoid at the final layer, the loss function is exactly the same as we saw in logistic regression:

$$L_{CE}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

Cross-Entropy Loss

If the neural network is a binary classifier with a sigmoid at the final layer, the loss function is exactly the same as we saw in logistic regression:

$$L_{CE}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

For multinomial classification

$$L_{CE}(\hat{y}, y) = - \sum_{i=1}^C y_i \log \hat{y}_i$$

If there is only one correct answer, where the truth is $y_i=1$, then this simplifies to be

$$L_{CE}(\hat{y}, y) = -\log \hat{y}_i$$

Plugging into softmax:

$$L_{CE}(\hat{y}, y) = -\log \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Computing the gradient

Logistic regression can be thought of as a network with just one weight layer and a sigmoid output. In that case the gradient is:

$$\begin{aligned}\frac{\partial LCE(w, b)}{\partial w_j} &= (\hat{y} - y) x_j \\ &= (\sigma(w \cdot x + b) - y) x_j\end{aligned}$$

But these derivatives only give correct updates for the last weight layer!

For deeper networks, computing the gradients requires looking back through all the earlier layers in the network, even though the loss is only computed with respect to the output of the network.

Solution: **error backpropagation algorithm**

Computation Graphs

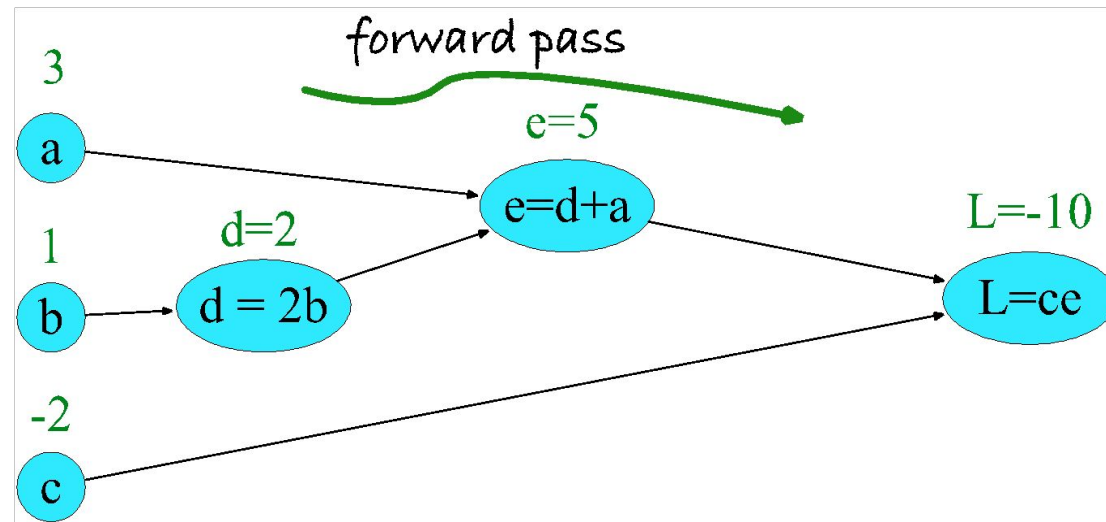
Although backpropagation was invented for neural nets, it is related to general procedure called **backward differentiation**, which depends on the notion of **computation graphs**.

A computation graph represents the process of computing a mathematical expression. The computation is broken down into separate operations. Each operation is a node in a graph.

$$L(a, b, c) = c(a + 2b)$$

$$\begin{aligned}d &= 2*b \\ e &= a + d \\ L &= c*e\end{aligned}$$

Forward pass



$$L(a, b, c) = c(a + 2b)$$

inputs $a = 3, b = 1, c = -2,$

$$\begin{aligned} d &= 2*b \\ e &= a + d \\ L &= c*e \end{aligned}$$

Backward differentiation

The importance of the computation graph comes from the backward pass, which is used to compute the derivatives that we'll need for the weight update.

How do we compute the derivative of our output function L with respect to the input variables a , b , and c ?

Backwards differentiation uses the **chain rule** from calculus.

$$\frac{\partial L}{\partial a'}, \frac{\partial L}{\partial b'}, \text{ and } \frac{\partial L}{\partial c}$$

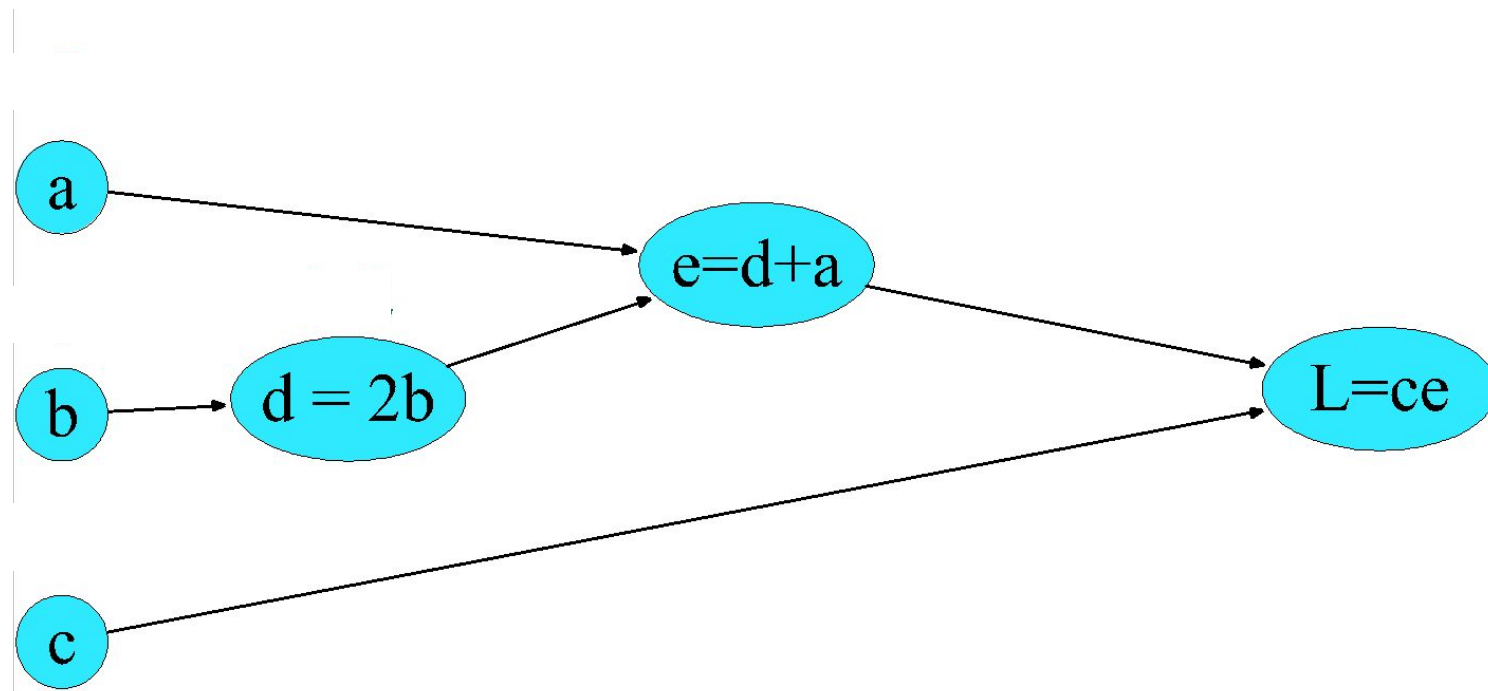
Chain rule

For a composite function $f(x) = u(v(x))$, the derivative of $f(x)$ is:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$$

Similarly for, $f(x) = u(v(w(x)))$, the derivative of $f(x)$ is:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx}$$

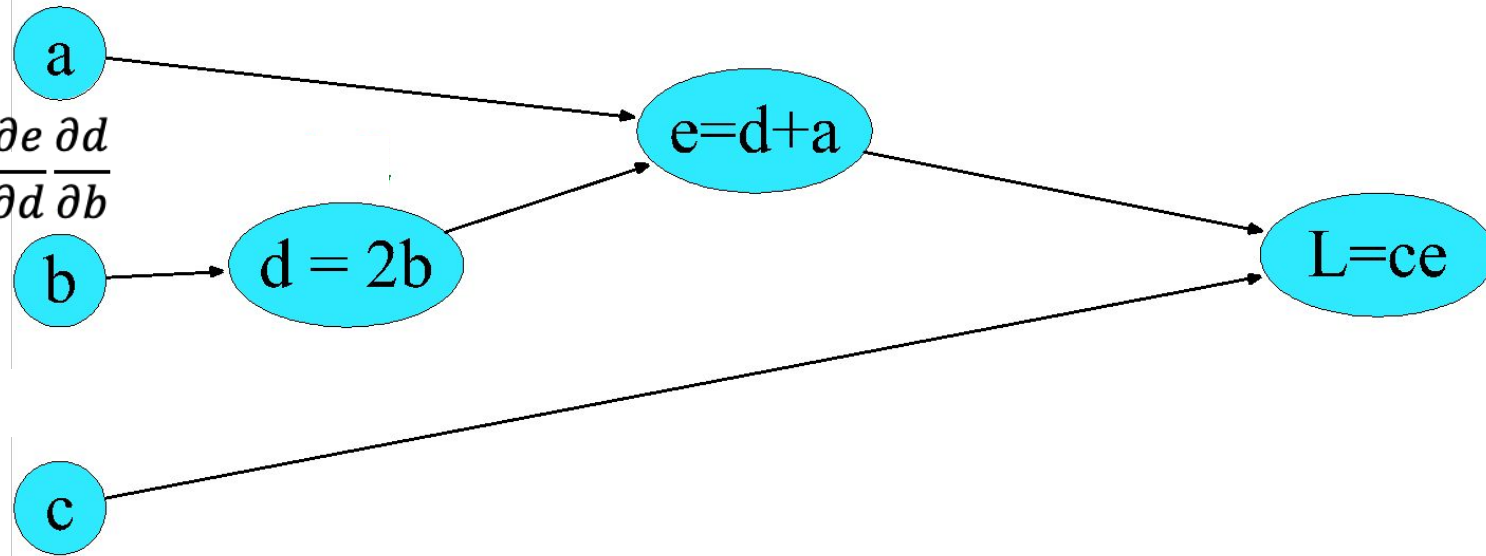


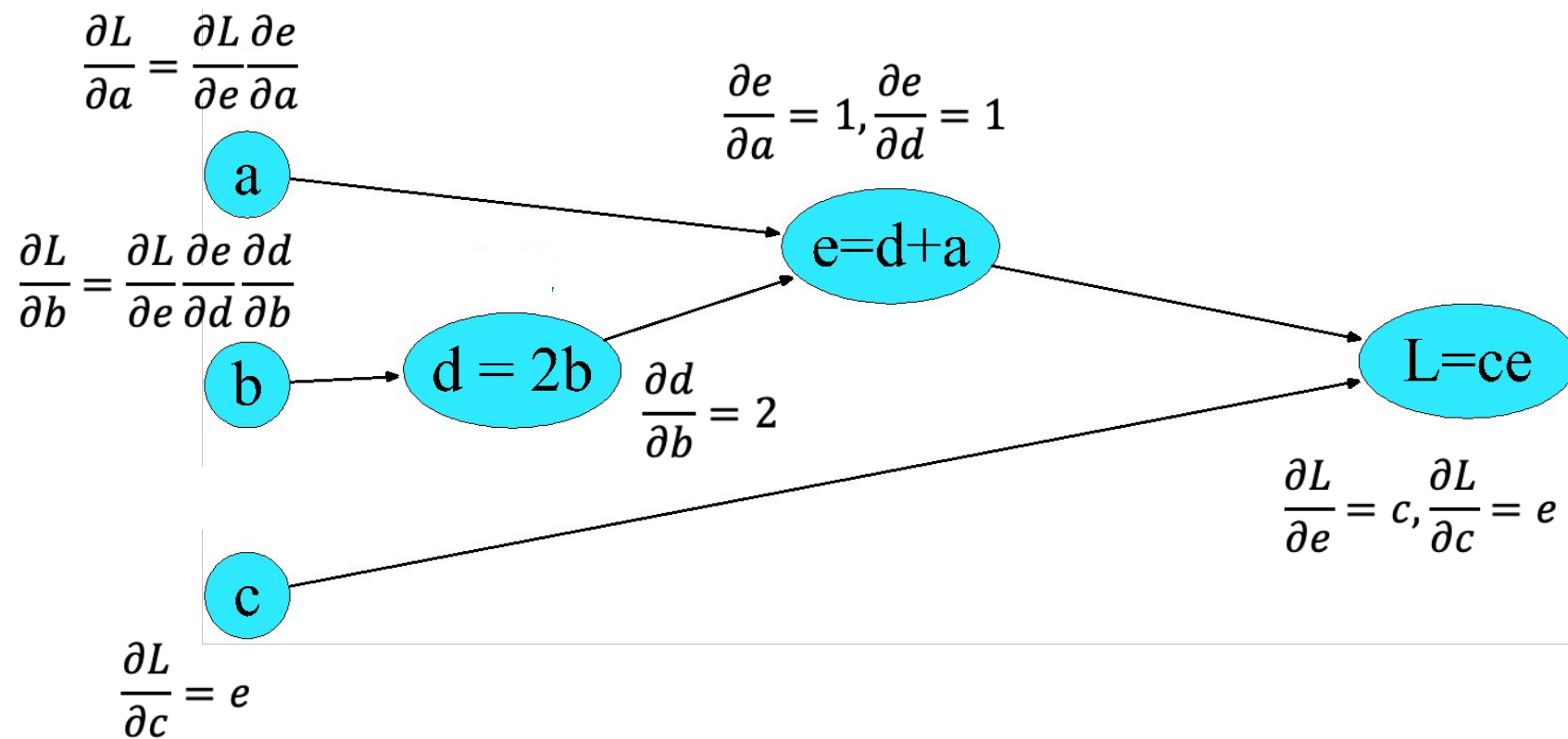
$$\frac{\partial L}{\partial c} = e$$

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a}$$

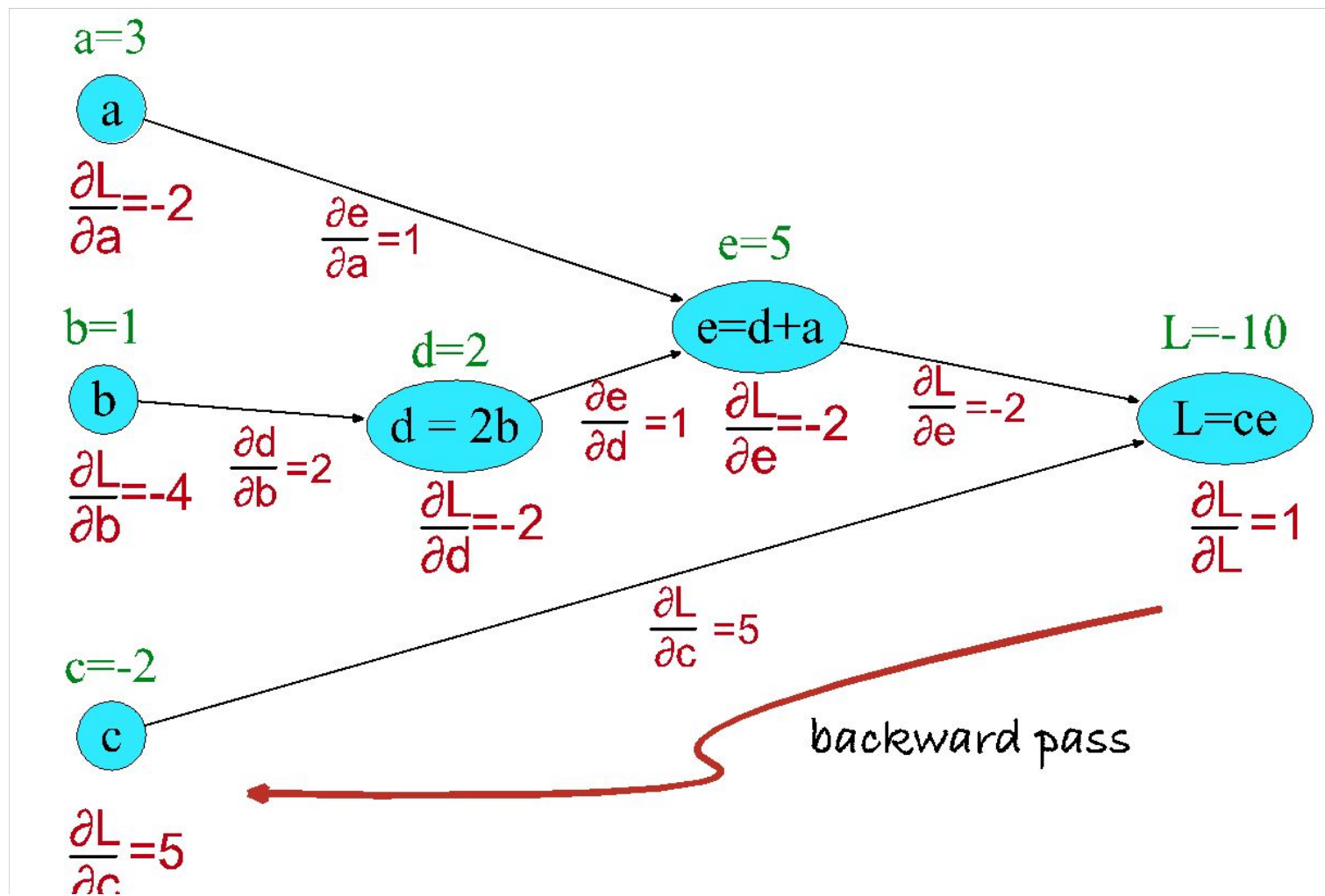
$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b}$$

$$\frac{\partial L}{\partial c} = e$$

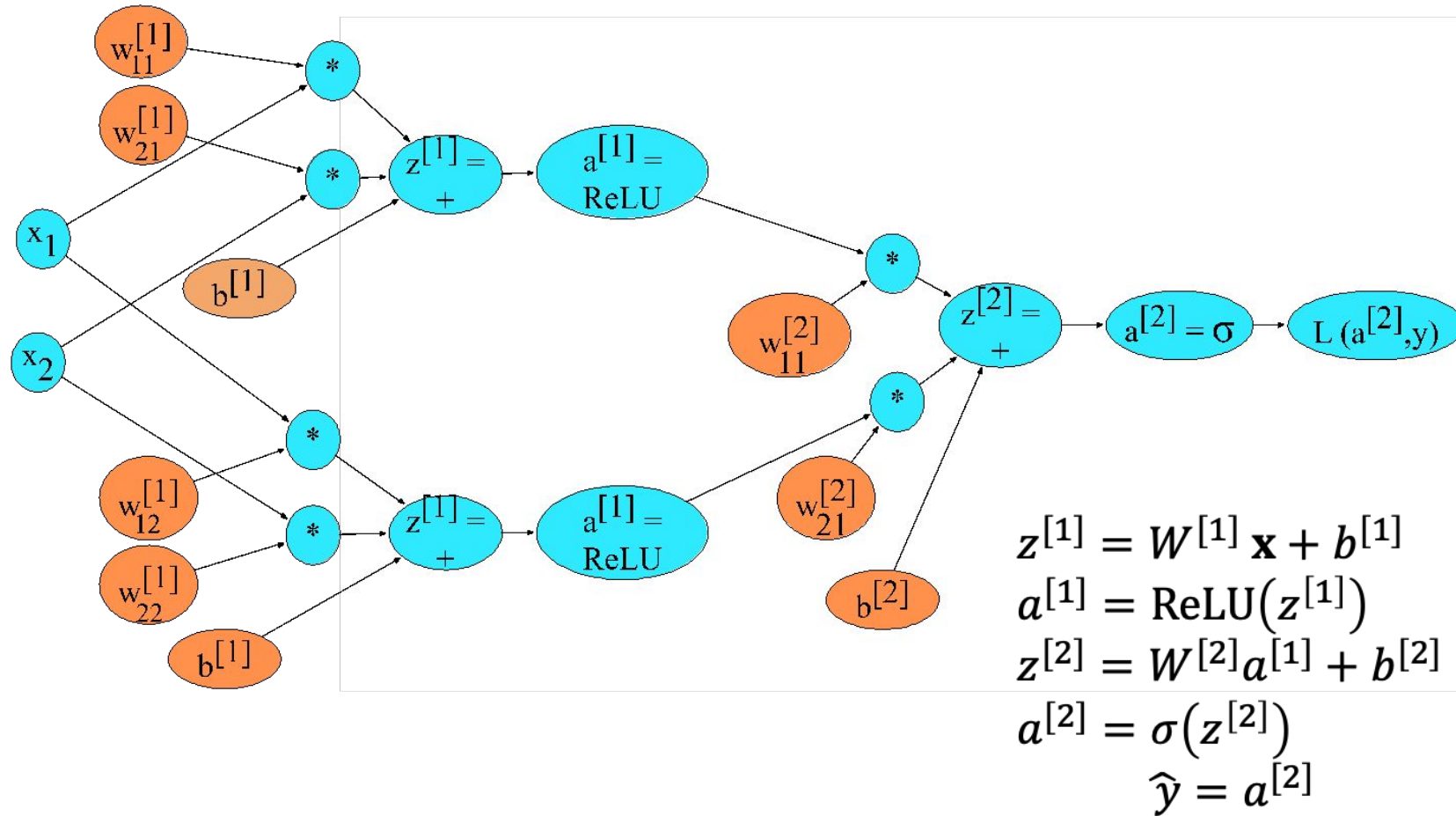




Backward pass



Computation Graph for a NN



Neural Language Models

Language Models

Estimate the probability of a sentence consisting of word sequence $w_{1:n}$

$$P(w_{1:n}) \approx \prod_{i=1}^n P(w_i | w_{i-k:i-1})$$

We need to estimate the probability of $P(w_{i+1} | w_{i-k:i})$ from a large corpus.

$$\hat{p}_{MLE}(w_{i+1} = m | w_{i-k:i}) = \frac{\#(w_{i-k:i+1})}{\#(w_{i-k:i})}$$

$$\hat{p}_{add-\alpha}(w_{i+1} = m | w_{i-k:i}) = \frac{\#(w_{i-k:i+1}) + \alpha}{\#(w_{i-k:i}) + \alpha|V|}$$

Neural Language Models

Neural Language models have several advantages over n-gram LMs:

1. They don't need smoothing
2. They can handle much longer histories.
3. They can generalize over contexts of similar words.
4. Neural LMs tend to have much higher predictive accuracy than n-gram LMs.

Disadvantage: slower to train than traditional n-gram LMs

Neural LMs

In neural LMs, the prior context is represented by embeddings. This allows neural LMs to generalize better than n-gram LMs.

Imagine in training we have seen

I have to make sure when I get home to feed the cat.

But we've never seen "feed the dog" in training. If in the test set, we want to predict the next word after

I forgot when I got home to feed the ____

An n-gram LM will only predict *cat*, but a neural LM will know that the embeddings for *cat* and *dog* are similar.

Neural LMs (Bengio et al 2003)

1. Associate each word in the vocabulary with a vector-representation, thereby creating a notion of similarity between words.
2. Express the joint probability *function* of a word sequence in terms of the word vectors for the words in that sequence.
3. Simultaneously learn the word vectors and the parameters of the *function*.

The word vectors are low-dimensional ($d=30$ to $d=100$) dense vectors, like we've seen before.

The probability function is expressed the product of conditional probabilities of the next word given the previous word, using a multi-layer, feed forward neural network.

Neural LMs

The input to the neural network is a k -gram of words $w_{1:k}$.

The output is a probability distribution over the next word.

The k context words are treated as a word window. Each word is associated with an **embedding** vector:

The input vector $\mathbf{x}$ just concatenates $v(w)$ for each of the k words:

$$v(w) \in \mathbb{R}^{d_w}$$

$$\mathbf{x} = [v(w_1); v(w_2); \dots; v(w_k)].$$

Neural LMs

The input $\mathbf{x}$ is fed into a neural network with 1 or more hidden layers:

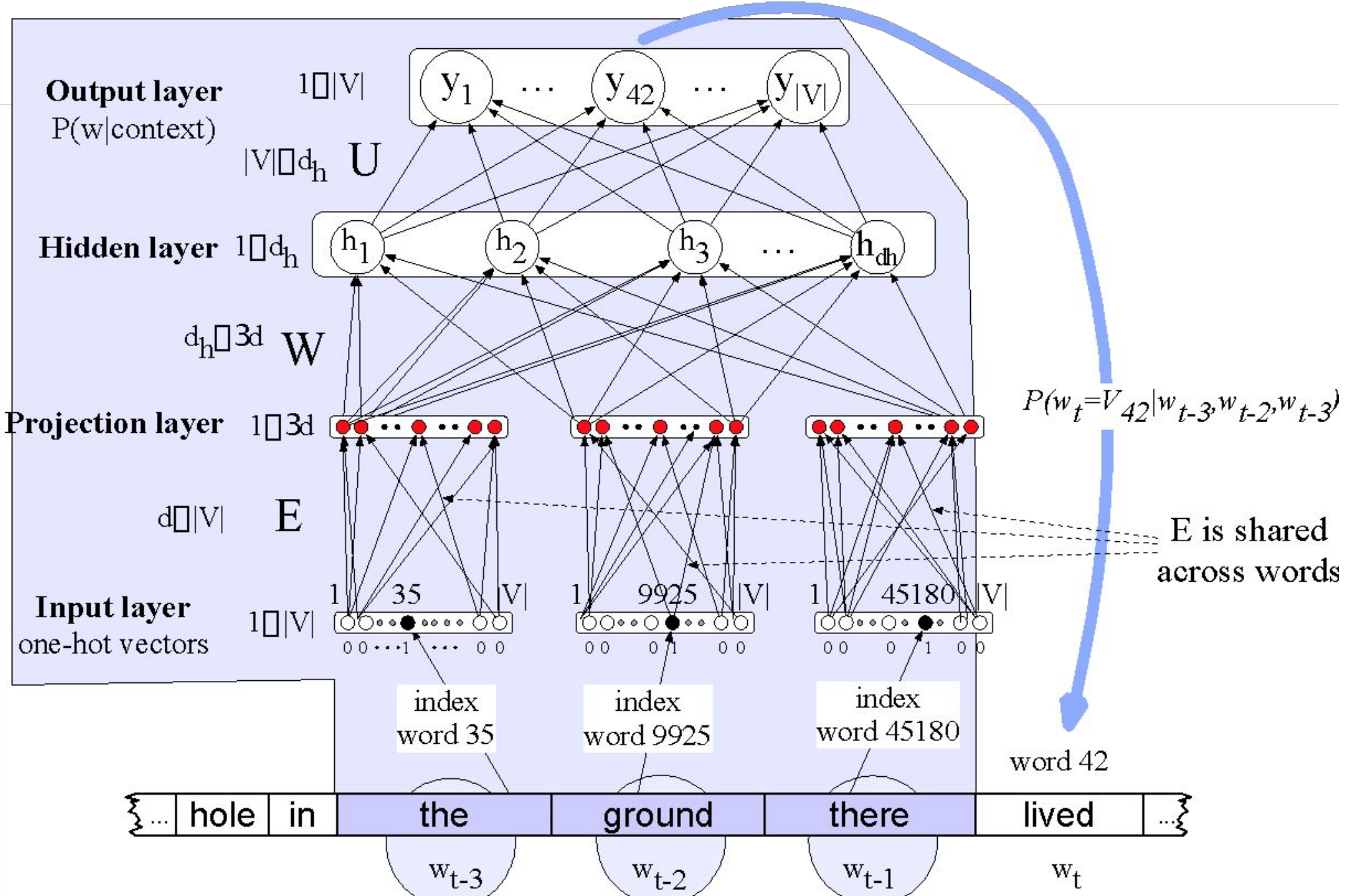
$$\hat{y} = P(w_i | w_{1:k}) = LM(w_{1:k}) = \text{softmax}(\mathbf{h} \mathbf{W}^2 + \mathbf{b}^2)$$

$$\mathbf{h} = g(\mathbf{x} \mathbf{W}^1 + \mathbf{b}^1)$$

$$\mathbf{x} = [v(w_1); v(w_2); \dots; v(w_k)]$$

$$v(w) = \mathbf{E}_{[w]}$$

$$w_i \in V \quad \mathbf{E} \in \mathbb{R}^{|V| \times d_w} \quad \mathbf{W}^1 \in \mathbb{R}^{k \times d_w \times d_{\text{hid}}} \quad \mathbf{b}^1 \in \mathbb{R}^{d_{\text{hid}}} \quad \mathbf{W}^2 \in \mathbb{R}^{d_{\text{hid}} \times |V|} \quad \mathbf{b}^2 \in \mathbb{R}^{|V|}$$



Training

The training examples are simply word k-grams from the corpus

The identities of the first $k+1$ words are used as features, and the last word is used as the target label for the classification.

Conceptually, the model is trained using cross-entropy loss.

Working with cross entropy loss works very well, but requires the use of a costly softmax operation which can be prohibitive for very large vocabularies, we we often use alternative loss functions or approximations.

Training

The training examples are simply word k-grams from the corpus

The identities of the first $k-1$ words are used as features, and the last word is used as the target label for the classification.

Conceptually, the model is trained using cross-entropy loss.

Advantages of NN LMs

Better results. They achieve better perplexity scores than SOTA n-gram LMs.

Larger N. NN LMs can scale to much larger orders of n . This is achievable because parameters are associated only with individual words, and not with n-grams.

They generalize across contexts. For example, by observing that the words *blue*, *green*, *red*, *black*, etc. appear in similar contexts, the model will be able to assign a reasonable score to the *green car* even though it never observed in training, because it did observe *blue car* and *red car*.

A by-product of training are **word embeddings**

Language Modeling

Goal: Learn a **function** that returns the joint probability

Primary difficulty:

1. There are too many parameters to accurately estimate. This is sometimes called the “curse of dimensionality”
2. In n-gram-based models we fail to generalize to related words / word sequences that we have observed.

Curse of dimensionality / sparse statistics

Suppose we want a joint distribution over 10 words.

Suppose we have a vocabulary of size 100,000.

$$100,000^{10} = 10^{50} \text{ parameters}$$

This is too high to estimate from data.

Chain rule



Probability tables

We construct tables to look up the probability of seeing a word given a history.

curse of	$P(w_t \mid w_{t-n} \dots w_{t-1})$
dimensionality	
azure	
knowledge	
oak	

The tables only store observed sequences.

What happens when we have a new (unseen) combination of n words?

Unseen sequences

What happens when we have a new (unseen) combination of n words?

1. Back-off
2. Smoothing / interpolation

We are basically just stitching together short sequences of observed words.

Alternate idea

Let's try **generalizing**.

Intuition: Take a sentence like

The cat is walking in the bedroom

And use it when we assign probabilities to similar sentences like

The dog is running around the room

A Neural Probabilistic LM

Bengio et al NIPS 2003

1. Use a vector space model where the words are vectors with real values $\mathbb{R}^m$. $m=30, 60, 100$. This gives a way to compute word similarity.
2. Define a function that returns a joint probability of words in a sequence based on a sequence of these vectors.
3. Simultaneously learn the word representations **and** the probability function from data.

Seeing one of the cat/dog sentences allows them to increase the probability for that sentence **and** its combinatorial # of “**neighbor**” **sentences** in vector space.

A Neural Probabilistic LM

Given:

A training set $w_1 \dots w_t$ where $w_t \in V$

Learn:

$$f(w_1 \dots w_t) = P(w_t | w_1 \dots w_{t-1})$$

Subject to giving a high probability to an unseen text/dev set (e.g. minimizing the perplexity)

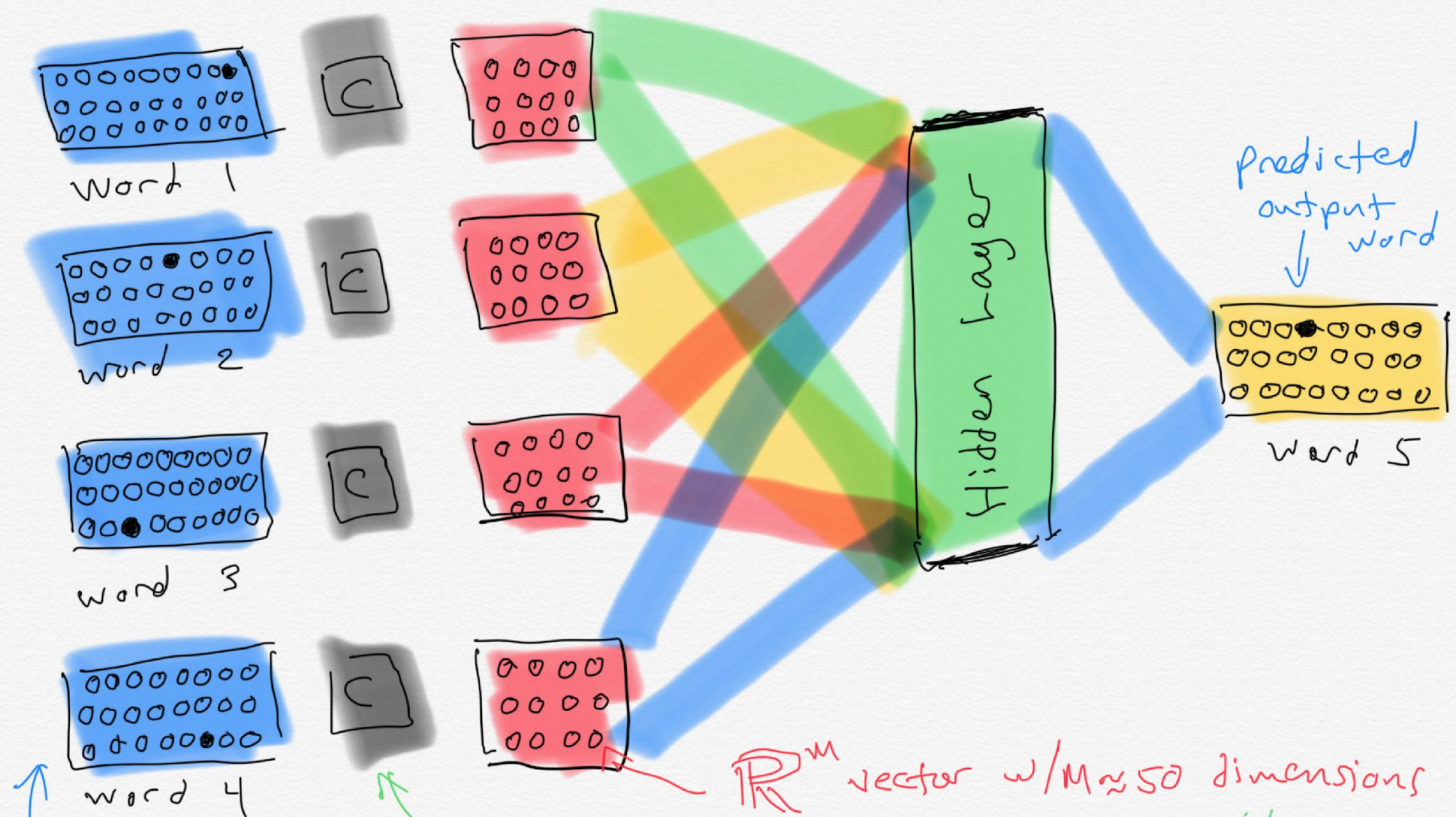
Constraint:

Create a proper probability distribution (e.g. sums to 1) so that we can take the product of conditional probabilities to get the joint probability of a sentence

A Neural Probabilistic LM

1. Create a mapping function C from any word in V onto $\mathbb{R}^M$. Store this in a V -by- M matrix. Initialize it with singular value decomposition (SVD).
2. The neural architecture: a function g maps sequence of word vectors onto a probability distribution over the vocabulary V

$$g(C(w_{t-n}) \dots C(w_{t-1})) = P(w_t | w_{t-n} \dots w_{t-1})$$

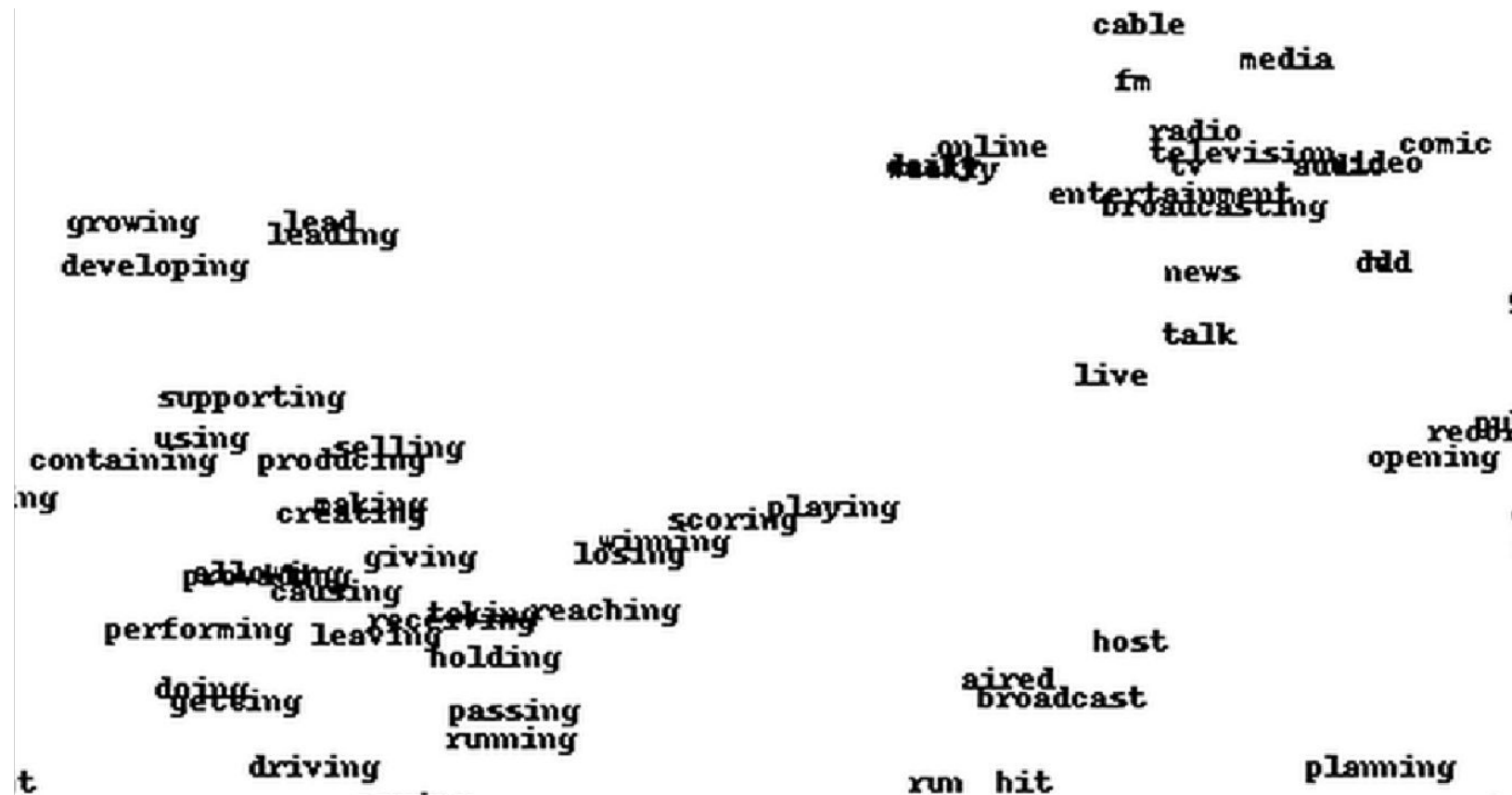


Input:
"1 hot vectors"
representing the identity of the input word

$\mathbb{R}^m$ vector w/ $m \approx 50$ dimensions
Mapping function from vocab item onto
a low dimensional dense vectors

Word embeddings

When the ~50 dimensional vectors that result from training a neural LM are projected down to 2-dimensions, we see a lot of words that are intuitively similar to each other are close together.



Current state of the art neural LMs

ELMo

GPT

BERT

GPT-2

